

5.2 Boundary Value Problem (Another Example)

In [14]: `using CairoMakie`

In the previous notebook, we showed examples of a relatively easy boundary value problem. There is only one derivative term. Usually, the left hand side of the equations have many more derivative terms. In this notebook, we will consider the following boundary value problem

$$\frac{d^2y}{dx^2} + \frac{2}{x} \frac{dy}{dx} - \frac{2}{x^2} y(x) = 0$$

This equation has 2nd derivatives, and first derivatives on the left hand side. We will solve the equation above the domain $x \in [1, 2]$ with the boundary conditions $y(1) = 5$ and $y(2) = 3$. In this case, the exact analytical solution is known and it is

$$y(x) = x + \frac{4}{x^2}$$

Specify parameters of the problem

In [15]: `Delta=0.001`
`x=1:Delta:2`
`α=5.0`
`β=3.0`

3.0

In [16]: `n=length(x)`
`A=zeros(n,n)`
`C=zeros(n)`

1001-element Vector{Float64}:

0.0
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0
 ⋮
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0

Set up the matrix system

So at every point in the domain, we will have

$$\frac{d^2 y}{dx^2} \approx \frac{y_{i-1} - 2y_i + y_{i+1}}{\Delta^2}$$

$$\frac{2}{x} \frac{dy}{dx} \approx \frac{2}{x_i} \frac{y_{i+1} - y_{i-1}}{2\Delta}$$

$$-\frac{2}{x^2} y \approx -\frac{2}{x_i^2} y_i$$

If we now sub everything back into the original equation, and assume we have a grid with 5 intervals (6 grid points), we can put into matrix form as

$$\begin{bmatrix} 1 & 0 & 0 & 0 \\ -1/(\Delta x_i) + 1/\Delta^2 & -2/\Delta^2 - 2/x_2^2 & 1/(\Delta x_i) + 1/\Delta^2 & 0 \\ 0 & -1/(\Delta x_i) + 1/\Delta^2 & -2/\Delta^2 - 2/x_3^2 & 1/(\Delta x_i) + 1/\Delta^2 \\ 0 & 0 & -1/(\Delta x_i) + 1/\Delta^2 & -2/\Delta^2 - 2/x_4^2 \\ 0 & 0 & 0 & -1/(\Delta x_i) + 1/\Delta \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

```
In [17]: A[1,1]=1.0
          C[1]=5.0
          for i=2:n-1
              A[i,i-1]=(1.0/Delta^2)-(2/(x[i]))*(1/(2*Delta))
              A[i,i]=-(2.0/Delta^2)-(2/(x[i]*x[i]))
              A[i,i+1]=(1.0/Delta^2)+(2/(x[i]))*(1/(2*Delta))
          end
          A[n,n]=1.0
          C[n]=3.0
```

3.0

```
In [18]: C
```

```
1001-element Vector{Float64}:
 5.0
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0
 0.0
 3.0
```

Solve the equation to get

In [19]: @time y=A\C

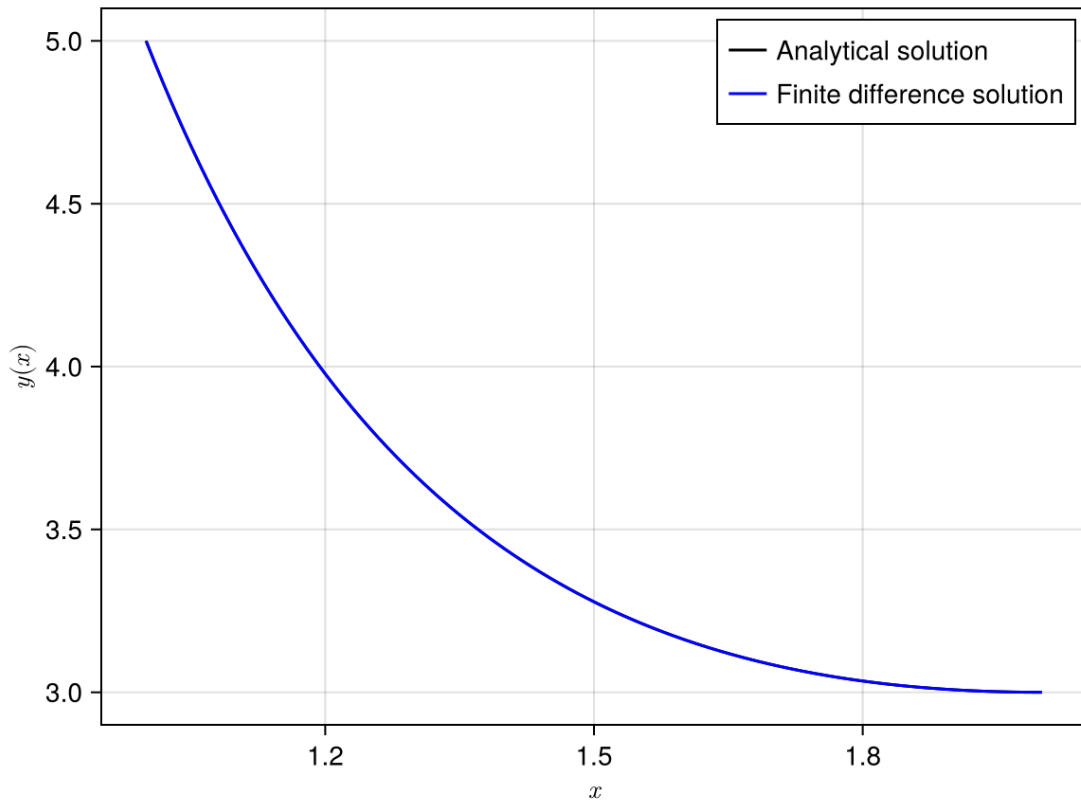
```
0.228910 seconds (9 allocations: 7.672 MiB, 96.73% gc time)
1001-element Vector{Float64}:
 4.99999999999996215
 4.993011985353724
 4.986047874979904
 4.979107573591243
 4.9721909863785125
 4.965298019003938
 4.958428577598386
 4.951582568758575
 4.944759899544302
 4.937960477475692
 4.93116188137326096
 4.9243632909397
 4.917564700505913
 4.910766110072026
 4.903967519638139
 4.897168929204252
 4.890370348770365
 4.883571768336478
 4.876773187902591
 4.869974607468704
 4.863176027034817
 4.85637744660093
 4.849578866167043
 4.842780285733156
 4.835981705299269
 4.829183124865382
 4.822384544431495
 4.815585963997608
 4.808787383563721
 4.801988803129834
 4.795190222695947
 4.78839164226206
 4.781593061828173
 4.774794481394286
 4.767995900960399
 4.761197320526512
 4.754398740092625
 4.747599159658738
 4.740799579224851
 4.733999998790964
 4.727200418357077
 4.72040083792319
 4.713601257489303
 4.706801677055416
 4.700002096621529
 4.693202516187642
 4.686402935753755
 4.679603355319868
 4.672803774885981
 4.666004194452094
 4.659204614018207
 4.65240503358432
 4.645605453150433
 4.638805872716546
 4.632006292282659
 4.625206711848772
 4.618407131414885
 4.611607550980998
 4.604807970547111
 4.598008390113224
 4.591208809679337
 4.58440922924545
 4.577609648811563
 4.570810068377676
 4.564010487943789
 4.557210907509902
 4.550411327076015
 4.543611746642128
 4.536812166208241
 4.530012585774354
 4.523213005340467
 4.51641342490658
 4.509613844472693
 4.502814264038806
 4.496014683604919
 4.489215103171032
 4.482415522737145
 4.475615942303258
 4.468816361869371
 4.462016781435484
 4.455217200991597
 4.44841762055771
 4.441618040123823
 4.434818459689936
 4.428018879256049
 4.421219298822162
 4.414419718388275
 4.407620137954388
 4.400820557520501
 4.394020977086614
 4.387221396652727
 4.38042181621884
 4.373622235784953
 4.366822655351066
 4.360023074917179
 4.353223494483292
 4.346423914049405
 4.339624333615518
 4.332824753181631
 4.326025172747744
 4.319225592313857
 4.31242601187997
 4.305626431446083
 4.298826851012196
 4.292027270578309
 4.285227690144422
 4.278428109710535
 4.271628529276648
 4.264828948842761
 4.258029368408874
 4.251229787974987
 4.2444302075411
 4.237630627107213
 4.230831046673326
 4.224031466239439
 4.217231885805552
 4.210432305371665
 4.203632724937778
 4.196833144503891
 4.190033564069904
 4.183233983636017
 4.17643440320213
 4.169634822768243
 4.162835242334356
 4.156035661900469
 4.149236081466582
 4.142436501032695
 4.135636920598808
 4.128837340164921
 4.122037759731034
 4.115238179297147
 4.10843859886326
 4.101639018429373
 4.094839437995486
 4.088039857561599
 4.081240277127712
 4.074440696693825
 4.067641116259938
 4.060841535826051
 4.054041955392164
 4.047242374958277
 4.04044279452439
 4.033643214090503
 4.026843633656616
 4.020044053222729
 4.013244472788842
 4.006444892354955
 4.000000000000000
 3.0
```

In [20]: yexact(x)=x+4/x^2

yexact (generic function with 1 method)

In [21]:

```
xplot=1:0.01:2
fig = Figure()
ax = Axis(fig[1, 1], xlabel = L"x", ylabel = L"y(x)")
lines!(ax,xplot,yexact.(xplot),color=:black,label="Analytical solution")
lines!(ax,x,y,color=:blue,label="Finite difference solution")
axislegend(ax,position=:rt)
fig
```



In [22]: `collect(x)`

1001-element Vector{Float64}:

```
1.0
1.001
1.002
1.003
1.004
1.005
1.006
1.007
1.008
1.009
⋮
1.992
1.993
1.994
1.995
1.996
1.997
1.998
1.999
2.0
```

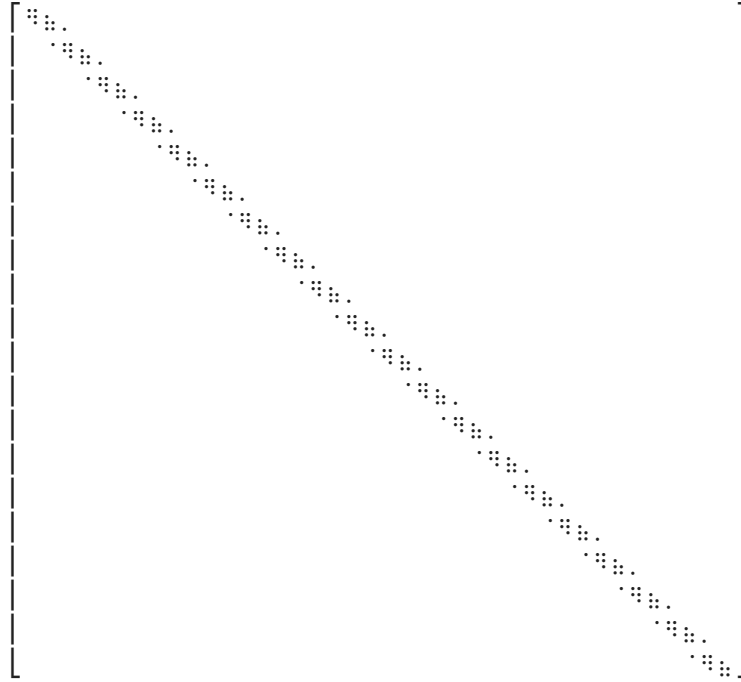
It takes a long time to solve this problem. Much faster to use `LinearSolve.jl` and tell Julia that you have a sparse matrix

In [23]: `using LinearSolve`
`using SparseArrays`

Define sparse Matrix

In [24]: `ASparse=sparse(A)`

1001×1001 SparseMatrixCSC{Float64, Int64} with 2999 stored entries:



In [25]: `probSparse=LinearProblem(ASparse,C)`
`@time y2=LinearSolve.solve(probSparse, UMFPACKFactorization())`

0.000349 seconds (104 allocations: 1.069 MiB)

retcode: Success

u: 1001-element Vector{Float64}:

```
5.0
4.993011985357482
4.986047874983633
4.979107573594945
4.972190986382188
4.965298019007587
4.95842857760201
4.951582568762174
4.944759899547878
4.937960477479248
⋮
3.000048258945858
3.000036923705416
3.0000271096507376
3.000018813732591
3.000012032909381
3.0000067641471255
3.000003004419435
3.0000007507074864
3.0
```

Note the time taken to solve the system using \ and using LinearSolve with sparse matrix definition. The gap gets bigger as you use larger number of grid points

In [26]: